



Princípios de Banco de Dados com Java e SQLite

Do JDBC às consultas parametrizadas — construa aplicações seguras, eficientes e profissionais com Java e SQLite.

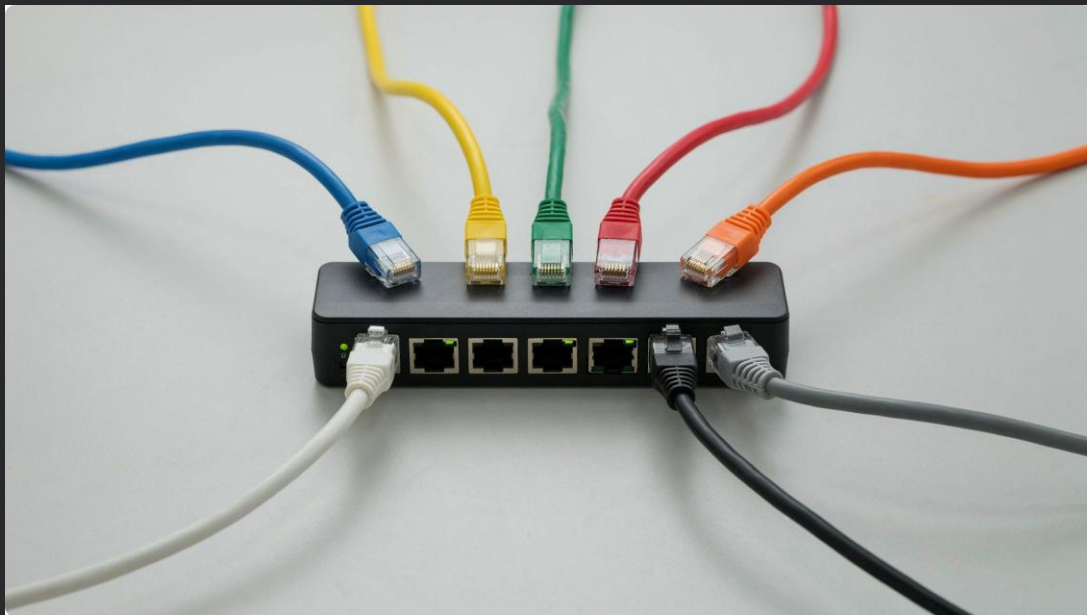
PARTE 1

O Alicerce — Conectividade com JDBC

Entenda como o Java se comunica com bancos de dados relacionais e por que o JDBC é o ponto de partida de toda a cadeia de persistência.



O que é o JDBC

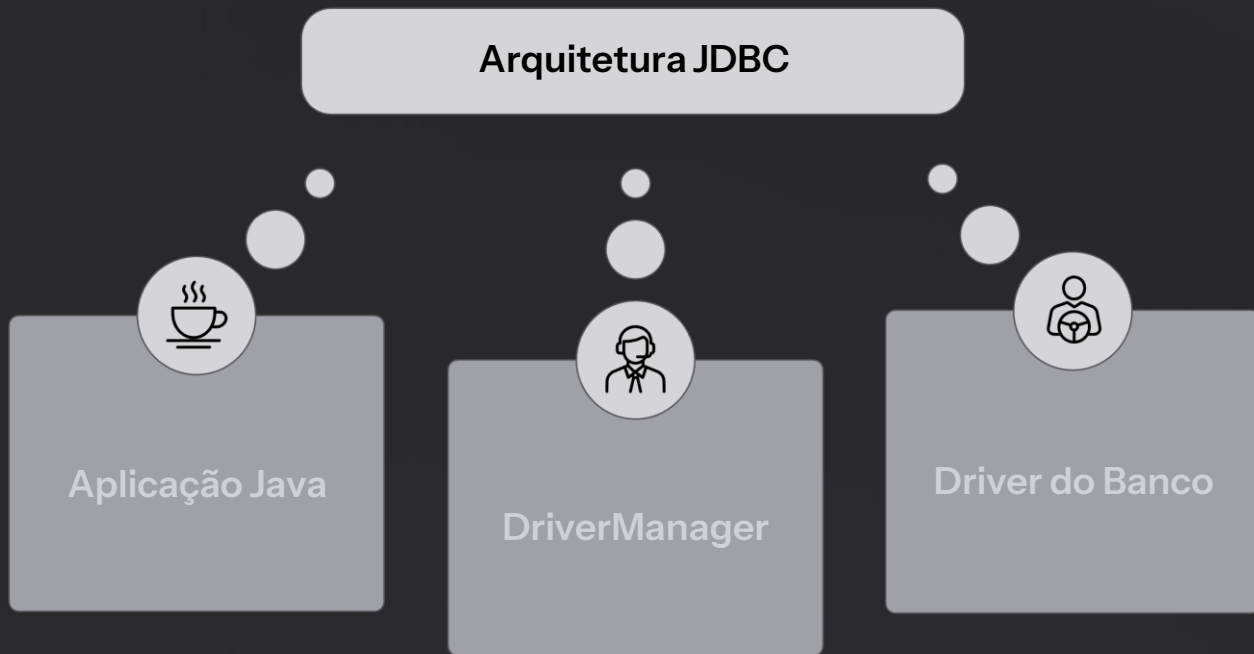


Java Database Connectivity

JDBC é a API padrão do Java para comunicação com bancos de dados relacionais. Ela define um conjunto de interfaces e classes que permitem executar SQL diretamente do código Java.

- ❏ A grande vantagem: o mesmo código-fonte pode acessar MySQL, PostgreSQL, SQLite ou qualquer outro banco apenas trocando o driver.

A Arquitetura JDBC



O **DriverManager** atua como mediador: a aplicação chama as interfaces padrão (`java.sql`), e o driver específico do banco traduz essas chamadas para o protocolo nativo.

O Papel do SQLite

O SQLite é um banco de dados relacional **leve, embutido e sem servidor**. Toda a base de dados fica em um único arquivo `.db` no disco — sem instalação de serviços externos.

- Ideal para aplicações embarcadas, móveis e desktop
- Perfeito para prototipação e desenvolvimento rápido
- Totalmente compatível com SQL padrão



Configurando o Ambiente

Adicione a dependência do driver SQLite-JDBC ao seu projeto. Com **Maven** ou **Gradle**, o processo é simples:

Maven

```
<dependency>
  <groupId>org.xerial</groupId>
  <artifactId>sqlite-jdbc</artifactId>
  <version>3.45.1.0</version>
</dependency>
```

Gradle

```
implementation 'org.xerial:sqlite-jdbc:3.45.1.0'
```

Por que gerenciadores de dependências?

Eles baixam, versionam e gerenciam automaticamente os JARs necessários, evitando conflitos manuais.

Estabelecendo a Conexão

A conexão com o SQLite é iniciada com uma única linha usando `DriverManager.getConnection()`.

```
String url = "jdbc:sqlite:meu_banco.db";  
Connection conn = DriverManager.getConnection(url);
```

jdbc:sqlite:

Protocolo que identifica o driver SQLite a ser usado

meu_banco.db

Nome do arquivo onde os dados serão armazenados

getConnection()

Método do DriverManager que retorna o objeto Connection



CAPÍTULO 2

Manipulando Dados

Com a conexão estabelecida, é hora de criar, consultar, atualizar e deletar registros utilizando as interfaces centrais do JDBC.

O Objeto Connection

Responsabilidades do Connection

- Representa uma sessão ativa com o banco de dados
- Cria objetos `Statement` e `PreparedStatement`
- Controla transações (commit/rollback)
- Deve ser fechado ao final para liberar recursos



Não fechar a conexão causa **vazamento de recursos** e pode travar o banco em aplicações de longa duração.

Padrão recomendado

```
try (Connection conn =  
    DriverManager.getConnection(url)) {  
    // operações aqui  
} // fechamento automático
```

Use **try-with-resources** para garantir o fechamento automático mesmo em caso de exceção.

A Interface Statement

O `Statement` é o objeto responsável por enviar comandos SQL ao banco de dados.

`executeQuery(sql)`

Executa consultas `SELECT` e retorna um `ResultSet` com os dados encontrados.

`executeUpdate(sql)`

Executa comandos `INSERT`, `UPDATE` e `DELETE`. Retorna o número de linhas afetadas.

`execute(sql)`

Versão genérica para DDL (`CREATE TABLE`, `DROP`). Retorna `true` se há `ResultSet` disponível.

O Objeto ResultSet



Navegando pelos resultados

O `ResultSet` funciona como um cursor que percorre linha a linha os resultados de um `SELECT`.

```
while (rs.next()) {  
    int id = rs.getInt("id");  
    String nome =  
    rs.getString("nome");  
}
```

- `next()` avança o cursor para a próxima linha
- Colunas acessadas por nome ou por índice (começando em 1)

Exemplo Prático: Listagem de Registros

Percorrendo a tabela `cursos` e exibindo todos os registros cadastrados:

```
String sql = "SELECT id, nome, carga_horaria FROM cursos";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql);

while (rs.next()) {
    int id = rs.getInt("id");
    String nome = rs.getString("nome");
    int carga = rs.getInt("carga_horaria");
    System.out.println(id + " | " + nome + " | " + carga + "h");
}
```

📄 O método `getString()` e `getInt()` mapeiam automaticamente os tipos SQL para os tipos Java correspondentes.



CAPÍTULO 3

Segurança e Performance

Aprenda a escrever consultas que protegem sua aplicação contra ataques e que executam com máxima eficiência.

O Problema da Injeção de SQL

Código vulnerável

```
String nome = request.getParam("nome");  
String sql = "SELECT * FROM usuarios  
WHERE nome = '" + nome + "'";
```

Se o usuário inserir ' OR '1'='1, a consulta retorna **todos os registros** — ignorando qualquer filtro.

Por que é perigoso?

- Expõe dados sensíveis de todos os usuários
- Permite deletar tabelas inteiras (DROP TABLE)
- Pode conceder acesso administrativo indevido
- É a vulnerabilidade #1 em aplicações web há décadas

⊗ Nunca concatene diretamente entradas do usuário em strings SQL.

PreparedStatement: A Solução

O `PreparedStatement` pré-compila a consulta no banco. Os parâmetros são tratados como **dados puros**, nunca como código SQL.

```
String sql = "SELECT * FROM usuarios WHERE nome = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, nomeDoUsuario);
ResultSet rs = pstmt.executeQuery();
```

Segurança

Parâmetros jamais são interpretados como SQL — ataques de injeção são impossíveis

Performance

A consulta é compilada uma vez e reutilizada, reduzindo o overhead em execuções repetidas

Legibilidade

Código mais limpo e manutenível, com clara separação entre estrutura e dados

Consultas Parametrizadas

Sem parâmetros



Sem parâmetros

Com parâmetros



Com parâmetros

Como os parâmetros funcionam

Cada `?` na query representa um espaço reservado. Os métodos `set*( )` preenchem esses espaços com valores tipados e escapados automaticamente.

- `setString(pos, valor)`
- `setInt(pos, valor)`
- `setDouble(pos, valor)`
- `setDate(pos, valor)`

Exemplo Prático: Inserção Segura

Cadastrando um novo curso de forma segura com `PreparedStatement`:

```
String sql = "INSERT INTO cursos (nome, carga_horaria) VALUES (?, ?)";

try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setString(1, "Java com JDBC");
    pstmt.setInt(2, 40);
    int linhasAfetadas = pstmt.executeUpdate();
    System.out.println("Inserido: " + linhasAfetadas + " registro(s)");
}
```

✓ O retorno de `executeUpdate()` confirma quantas linhas foram afetadas — útil para validar o sucesso da operação.



CAPÍTULO 4

Avançando no Desenvolvimento

Domine transações, paginação, integridade referencial e boas práticas arquiteturais para construir aplicações robustas e escaláveis.

Tratamento de Exceções

A classe SQLException

`SQLException` é a exceção raiz do JDBC. Ela carrega informações valiosas para diagnóstico:

- `getMessage()` — mensagem legível do erro
- `getSQLState()` — código de estado padrão SQL
- `getErrorCode()` — código específico do fornecedor

Padrão recomendado

```
try (Connection conn = ...) {  
    // operações  
} catch (SQLException e) {  
    System.err.println(  
        "Erro: " + e.getMessage());  
    System.err.println(  
        "SQLState: " + e.getSQLState());  
}
```

Transações no JDBC

Transações garantem que um conjunto de operações seja executado **completamente ou não seja executado** — garantindo consistência dos dados.

```
conn.setAutoCommit(false); // desativa commit automático
try {
    stmt.executeUpdate("INSERT INTO pedidos ...");
    stmt.executeUpdate("UPDATE estoque ...");
    conn.commit(); // confirma ambas as operações
} catch (SQLException e) {
    conn.rollback(); // desfaz tudo em caso de erro
}
```

commit()

Confirma permanentemente todas as alterações da transação

rollback()

Desfaz todas as alterações, restaurando o estado anterior

setAutoCommit(false)

Agrupa operações para controle manual da transação

Paginação de Resultados

O problema

Carregar milhares de registros de uma vez consome memória e degrada a experiência do usuário. A paginação resolve isso entregando os dados em lotes.

A solução com SQLite

```
SELECT * FROM cursos
ORDER BY nome
LIMIT 10 OFFSET 20;
```

`LIMIT` define quantos registros retornar. `OFFSET` define a partir de qual registro começar.

Cálculo da página

```
int pagina = 3;
int tamanhoPagina = 10;
int offset = (pagina - 1) * tamanhoPagina;
```

```
String sql =
    "SELECT * FROM cursos " +
    "LIMIT ? OFFSET ?";
pstmt.setInt(1, tamanhoPagina);
pstmt.setInt(2, offset);
```

Integridade Referencial



Chaves Estrangeiras no SQLite

Por padrão, o SQLite **não impõe** chaves estrangeiras. É necessário habilitá-las explicitamente a cada conexão:

```
SQLiteConfig config = new  
SQLiteConfig();  
config.enforceForeignKeys(true);  
Connection conn =  
DriverManager.getConnection(  
    url, config.toProperties());
```

Com isso ativado, operações que violem relacionamentos entre tabelas lançam `SQLException` automaticamente.

Boas Práticas e Arquitetura

Separe as responsabilidades do seu sistema para facilitar testes, manutenção e evolução do código.



Padrão DAO

Data Access Object isola toda a lógica de acesso ao banco. As regras de negócio nunca conhecem SQL diretamente.



Interfaces

Defina contratos com interfaces (CursoDAO) para facilitar a troca de implementações e a criação de mocks em testes.



Reutilização de Conexões

Use um único ponto de criação de conexões (Factory ou Singleton) para evitar conexões duplicadas e facilitar a troca de banco.

Conclusão e Próximos Passos

O JDBC é o alicerce de toda a persistência no ecossistema Java. Dominar seus conceitos torna qualquer ORM mais transparente e seu código mais seguro.

1 Consolide o JDBC puro

Implemente um CRUD completo com SQLite: crie tabelas, insira, consulte, atualize e delete usando `PreparedStatement`.

2 Aplique o padrão DAO

Refatore seu código separando o acesso a dados da lógica de negócio. Crie interfaces e implemente com SQLite.

3 Evolua para ORMs

Com a base sólida, explore Hibernate ou Spring Data JPA — e entenda o que acontece "por baixo dos panos".

